The DevOops Handbook

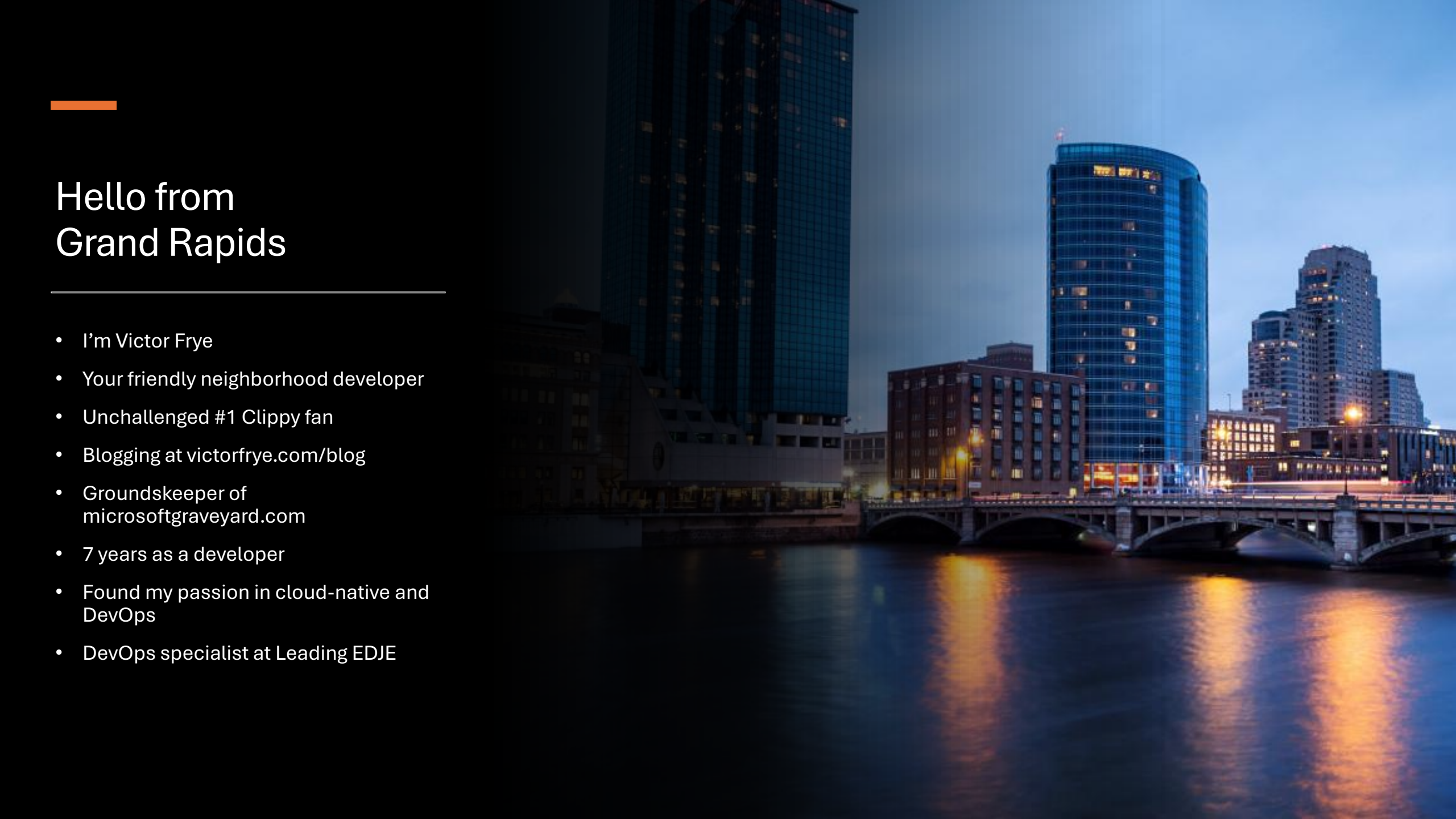
How to guarantee slow software delivery

Agenda

- Introduction
- DevOps Overview
- DevOops Disclaimer
- Case study
- Closing remarks
- Q&A

Hello from Grand Rapids

- I’m Victor Frye
- Your friendly neighborhood developer
- Unchallenged #1 Clippy fan
- Blogging at victorfrye.com/blog
- Groundskeeper of microsoftgraveyard.com
- 7 years as a developer
- Found my passion in cloud-native and DevOps
- DevOps specialist at Leading EDJE

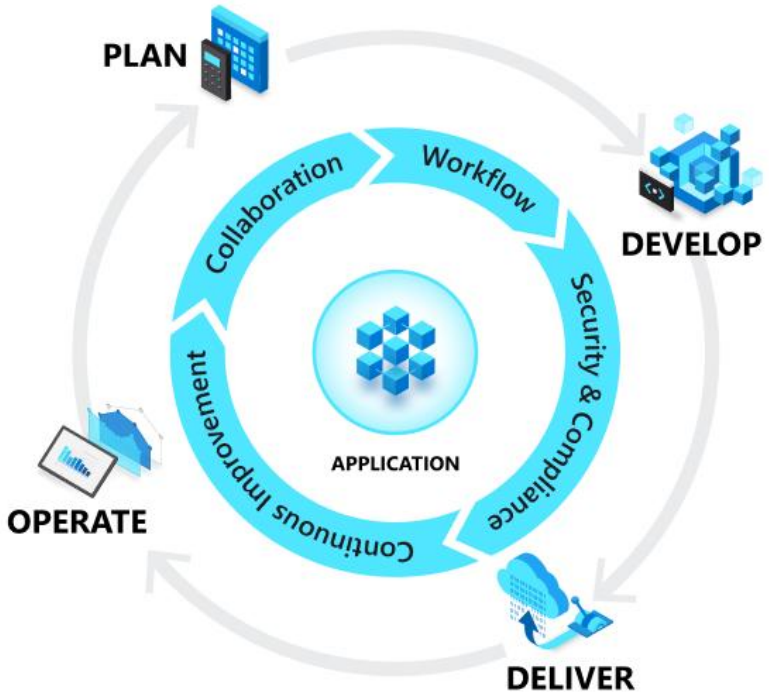


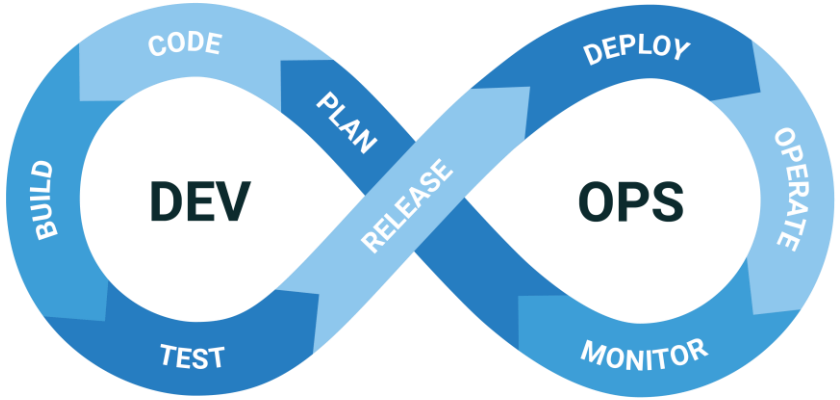
Leading EDJE

- Custom software consultancy
  - Positive disruption
  - Boutique mindset
  - Outcome focused
- ~75 employees
- Real. Fun. Geeks.
- Free lunch and learns
- Expert talent
  - Cloud
  - DevOps
  - Web
  - Agile
  - AI

DevOps and the culture problem

Application Lifecycle

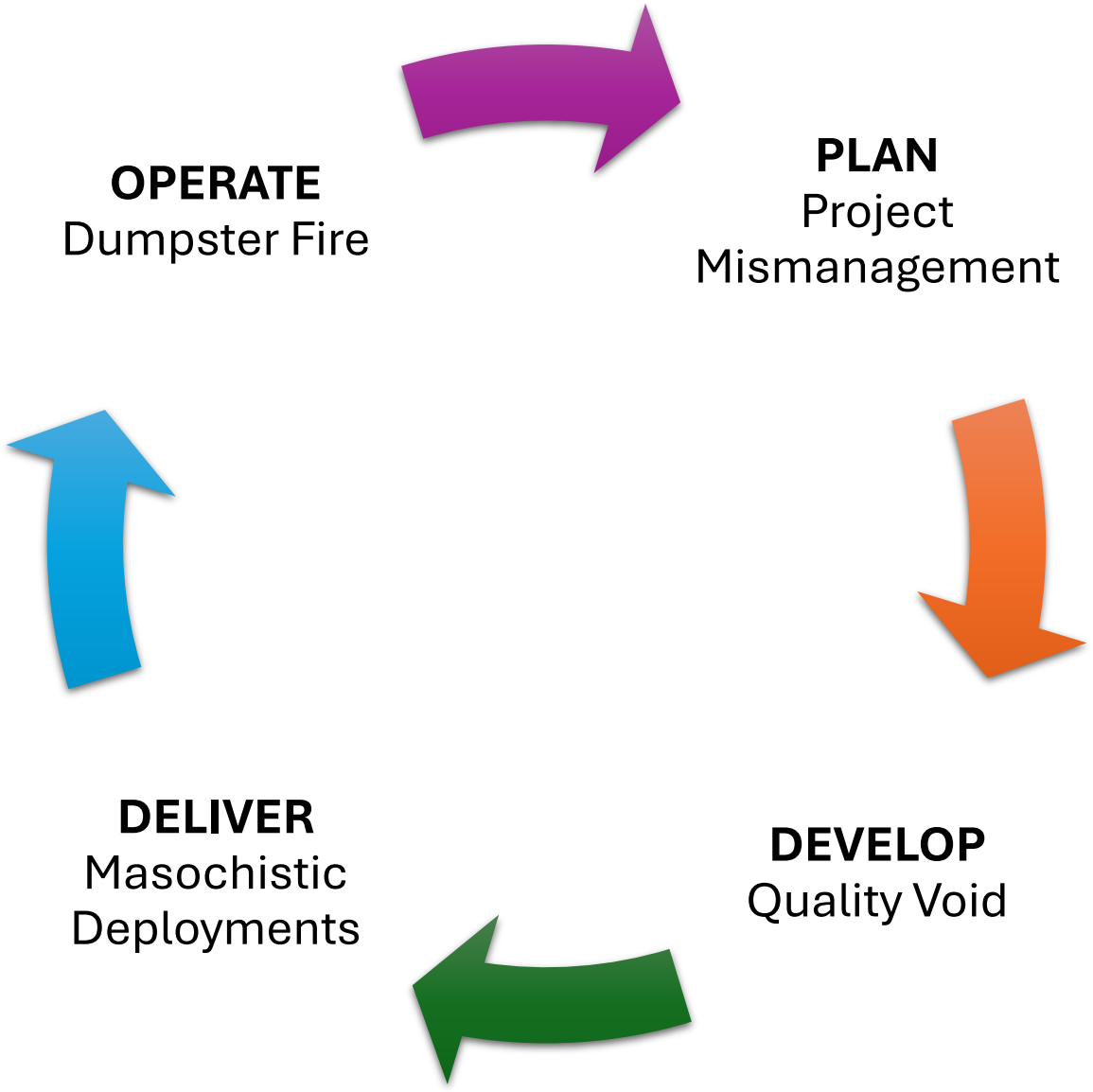




Continuous Lifestyle

- Continuous Planning
- Continuous Integration
- Continuous Delivery
- Continuous Operations
- Continuous Quality
- Continuous Security
- Continuous Improvement
- Continuous Feedback

Continuous Failure



Our DevOops Disclaimer

- This is satirical and in jest
- Based on real experiences
- Laugh and groan and cry
- Continuous improvement
- Hear great ideas from other speakers

What is DevOops?

The Diff

DevOps

The primary goal of DevOps is to streamline and automate the application lifecycle to accelerate the delivery of high-quality products.

DevOops

The primary goal of DevOops is to remove risk, introduce conformity, and accelerate the application lifecycle to guarantee slow software delivery.

The Goals

We want to accelerate our time to market

By embracing the quality void

By prioritizing everything as number one

We want to adapt to the market and competition

But we want to tightly control work for alignment

By moving at a break-neck speed

We want to maintain system stability and reliability

By ensuring changes are nigh impossible

By documenting everything

We want to improve the mean time to recovery

By keeping recovery time to zero

By hiding our mistakes

Case Study: SloweTech Corporation

Our Current Workplace

DELIVER
Deployments

Continuous integration and continuous delivery

Manual RFC for production changes

Some teams deploy during the day and some during the evening

PLAN
Project Management

Scrum with 2-week sprints

Average team consists of:

- 1 scrum master
- 1 business analyst
- ~3 developers

Semi-cross functional skillsets

DEVELOP
Quality Control

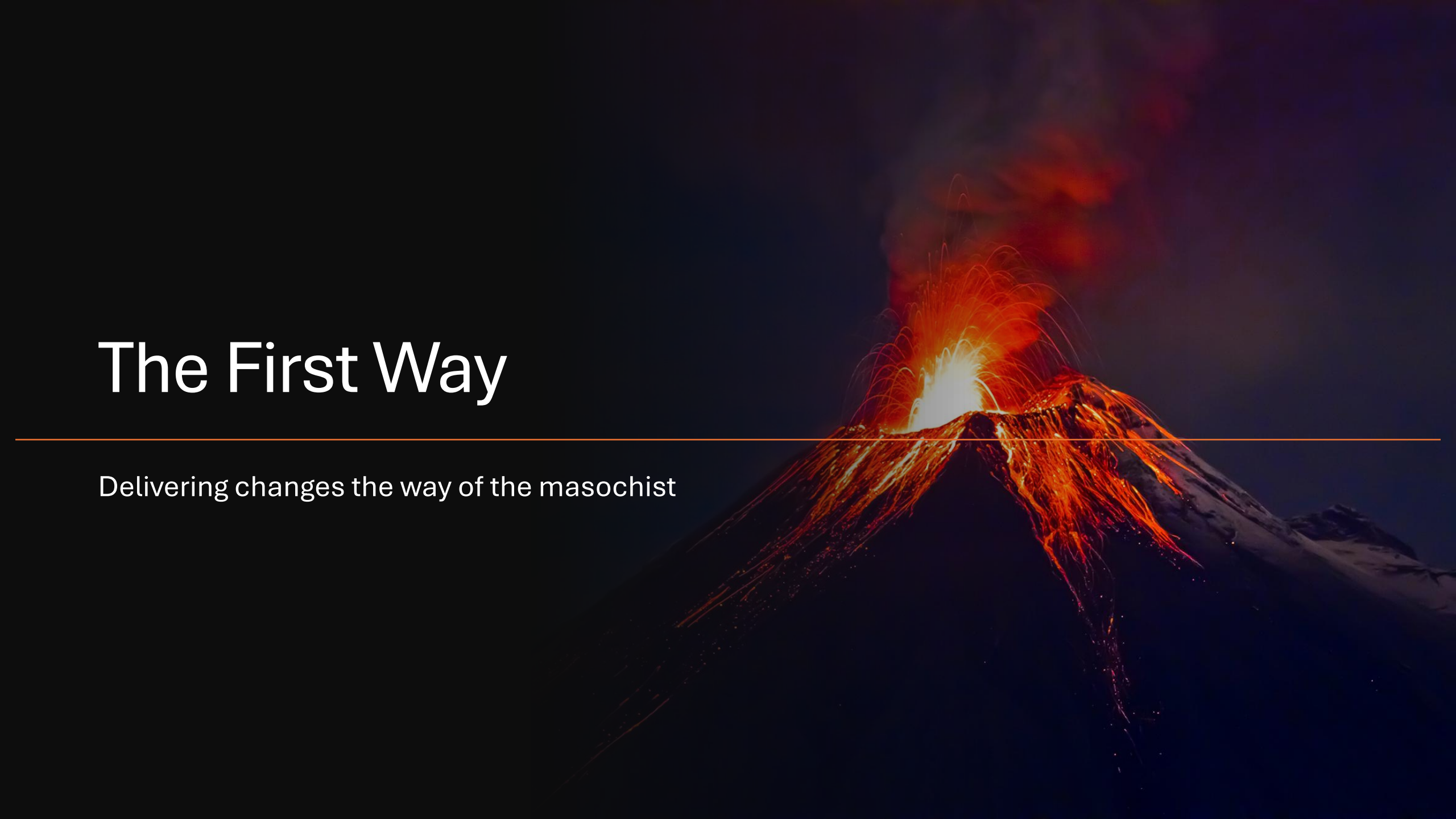
Quality engineers own testing

Unit Tests are optional

Static code analysis on CI

The First Way

Delivering changes the way of the masochist



Afterhours deployments

- Something went wrong with core LoB app
- Downtime experienced by customers
- We looked *really bad* to the business
- Any night is fine, except maybe Fridays?

Monday night only

- Things keep going wrong still
- Align all changes to one time window
- If something breaks, we know it’s because we changed stuff
- Stay in control and do not look bad in front of the business

Changes to RFC Approvals

Create RFC

- Must be completed two weeks prior to approval
- Include business justification and expected changes

Dev Done

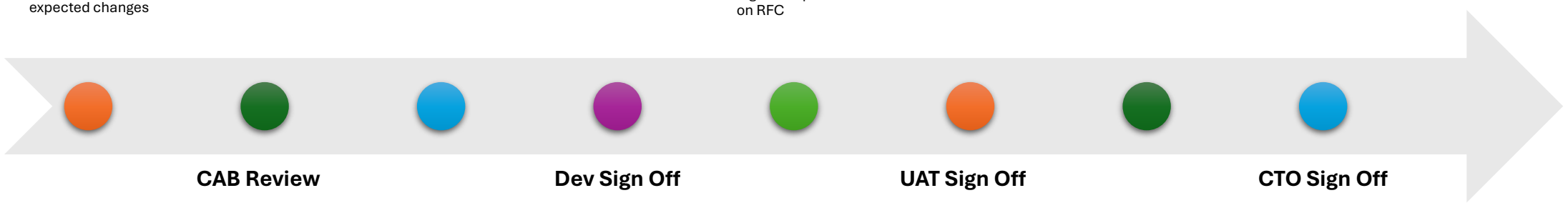
- Work is all merged to main
- All cards are ready for testing
- Link all tasks to RFC

QE Sign Off

- Test every change
- Expected turnaround of one minute
- Signature provided on RFC

Director Sponsorship

- Provide signatures and RFC document to director



CAB Review

- Review all scheduled changes 1-by-1
- Mandatory, all teams represented by one person
- CTO optional

Dev Sign Off

- Another team member
- Preferably the dev lead

UAT Sign Off

- Demo all work to stakeholders on call
- Thumbs up or down
- Signature provided on RFC

CTO Sign Off

- Owning director present RFC to me
- No one else is allowed to ask

Offshores team owns deployment

- New offshores team
- Opposite time zone
- Works off core business hours
- Manages all deployments

Common themes of painful deployments

Manual

Fear

Code Freezes

Downtime

Recover fast

Feature Flags

Deploy code, release features

Recover fast

Monitoring

System alerts

Health checks

Know it failed before anyone else

Release strategies

Blue/Green deployments

Canary

API Versioning

Backups

The Second Way

Conformational planning by mismanagement

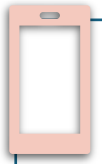
Process conformity

- The 3 questions in daily standup
- Predefined planning, review, retro
- Board layout is the same for all teams
- Story points for consistent estimation

“Just get it done” prioritization

- Simplified one-point prioritization scale
- Must hit deadlines at all costs

Silo team skillsets

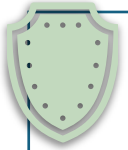


Frontend teams

Backend teams

Database team

Quality team



Security team

Infrastructure team

Support team

DevOps team

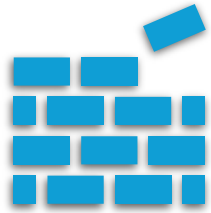
Documentation required

- Wiki is out of date
- Xmldocs everywhere
- Knowledge retention vs employee retention
- Do not slow down to document

Common themes of process conformity

Control

Promises



Consistency

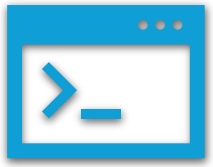
Improve the inner loop

Cycle time

Time in queues

Code on the shelf

Track unplanned work



Developer Experience

Make local dev painless

New member up to speed

PR open-close average time

Mob programming

Dev coffee chat

Post standup vs anytime

Completely optional

The Third Way

Rapid development through the quality void

Microservice nexus

- We have microservices!!
- Single database server
- Cascading failures are a feature
- Application expects system perfection
- Development speed starts at architecture

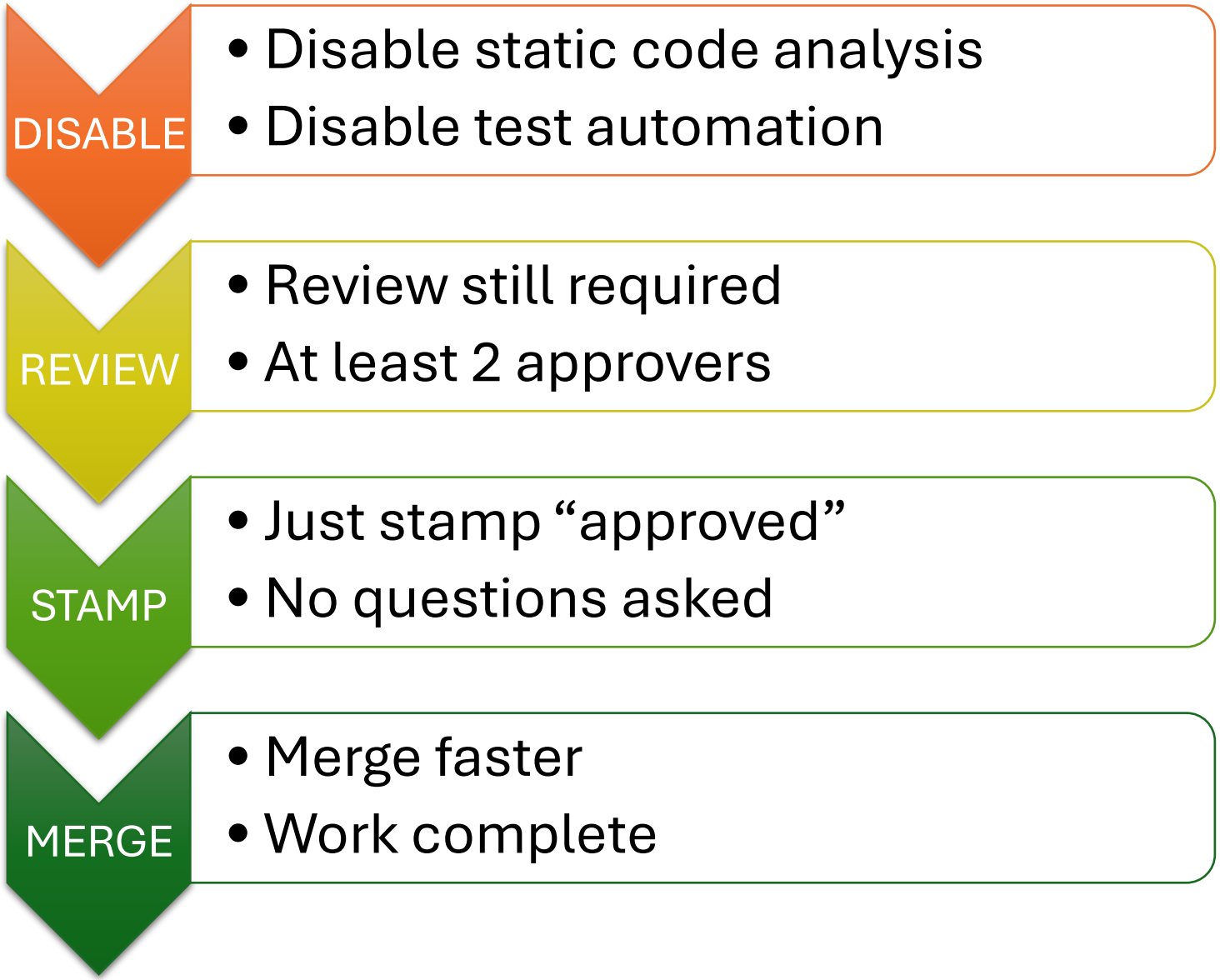
More code is always better

- `if true && !false => return true`
- Use that “is-true” package
- Put that plaintext secret in source
- The AI is wise, let’s trust it

Collapse the test pyramid

- Tests cost time and effort
- They break when we make changes
- Still have bugs with new features
- Just dump tests and use users?

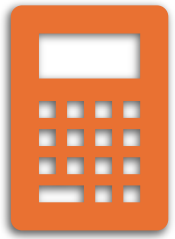
Rubber stamp pull requests



Security is optional

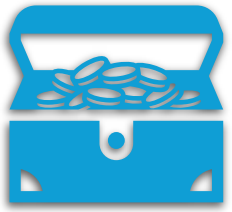
- Meeting security requirements is time consuming
- 5k line vulnerability report
- Only see security team when there’s a problem
- We can secure it later

Common themes of skipping quality



Accountability

Speed



Value

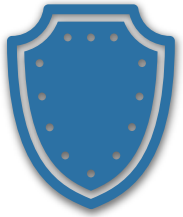
Shift stability left

Automate testing

In pipeline workflows

Get the QE to help

Slow dev? Git Gud



Security matters

DevSecOps

Vulnerability scanning

Keep secrets out of code

Rework Rate

Releases for bugs/missed reqs?

Track unplanned work

Our DevOops Workplace

DELIVER
Masochistic Deployments

Laborious RFC process

Still cannot get an incident free release

IT spend is up due to duplicate teams overseas

PLAN
Project Mismanagement

Isolated teams

No ownership of developer experience

Always in a rush

DEVELOP
Quality Void

No tests anywhere

Active ransomware attack

Do you have some spare bitcoin?

OPERATE DUMPSTER FIRE

OR NOT?

Does not have to be this way

Embrace DevOps culture and practices

Favor continuous improvement

Consult with experienced experts

Thank you

Victor Frye

616-706-7407

victorfrye@outlook.com

victor.frye@leadingedje.com

leadingedje.com

victorfrye.com

linkedin.com/in/victorfrye
github.com/victorfrye